



## Modul 2. Design Principle (S.O.L.I.D.)

### 2.1 Pendahuluan

SOLID merupakan kumpulan dari beberapa principle yang diwujudkan oleh engineer-engineer yang ahli dibidangnya. SOLID membantu kita mengembangkan sebuah perangkat lunak dengan tingkat kekukuhan yang tinggi.

Pada dasarnya OOP dan SOLID merupakan 2 (dua) hal yang berbeda. OOP adalah sebuah paradigma untuk menuliskan program yang sudah diadaptasi oleh beberapa bahasa pemrograman, sedangkan SOLID merupakan sebuah principle dalam pengembangan perangkat lunak. Ketika sudah berhasil menulis kode dengan mengikuti paradigma OOP, bukan berarti sudah mengikuti suatu *design principle* dalam *coding*.

Dalam mengembangkan sebuah perangkat lunak, jika bisa menuliskan kode dengan bentuk dan ukuran yang sama seperti halnya kumpulan bola di atas, dapat menciptakan sebuah sistem yang kukuh. Dengan terciptanya sistem yang kukuh, dapat lebih mudah dan leluasa mengganti komponen dan memperluas sistem tanpa adanya gangguan.

#### 2.1.1 Tujuan SOLID

SOLID bukanlah suatu hukum atau aturan tertentu yang wajib dipatuhi, melainkan sebuah prinsip yang dimaksudkan untuk membantu dalam menuliskan kode yang rapi. Berikut adalah tujuan dari prinsip SOLID dalam pembuatan struktur *mid-level* perangkat lunak:

- Toleran terhadap perubahan.
- Mudah dipahami.
- Komponen dasar dapat digunakan kembali dalam bentuk *software system* lainnya.

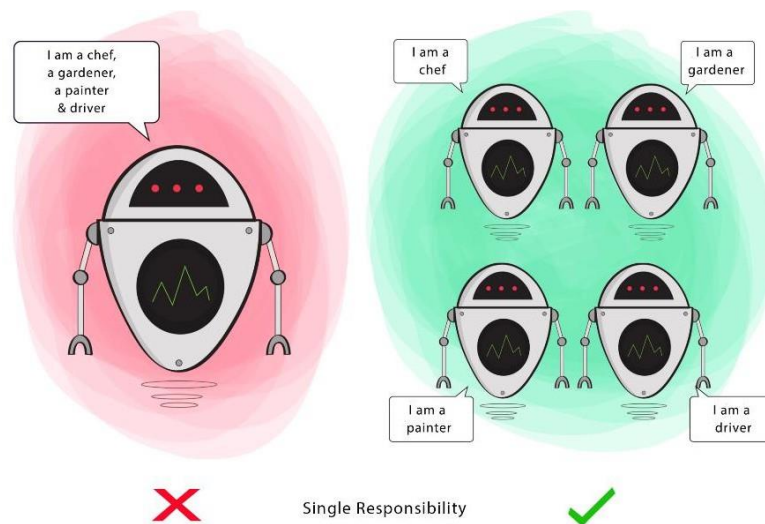
Istilah *mid-level* yang merujuk pada prinsip SOLID ini diterapkan oleh engineer yang bekerja pada level module. Prinsip ini diterapkan tepat di atas level kode. Manfaatnya, dapat membantu menentukan jenis struktur perangkat lunak yang digunakan dalam modul dan komponen. Setelah komponen tersebut dapat didesain dengan baik menggunakan prinsip SOLID, maka selanjutnya dapat melanjutkan ke dalam prinsip-prinsip arsitektur tingkat tinggi (*high-level architecture*).

### 2.1.2 Single Responsibility Principle (SRP)

Tanggung jawab (responsibility) berarti bahwa jika suatu *class* punya 2 (dua) fungsionalitas yang tak memiliki keterkaitan untuk melakukan suatu perubahan, maka kita harus membagi fungsionalitas yang berbeda tersebut dengan cara memisahnya menjadi dua *class* yang berbeda.

Maka dari itu, setiap *class* yang sudah dipisahkan berdasarkan fungsionalitasnya hanya akan menangani satu tanggung jawab. Lebih lanjut, jika kita melakukan perubahan tanggung jawab, kita tinggal fokus pada masing-masing *class* yang sudah dipisahkan tersebut.

Berikut adalah gambaran ilustrasi dari SRP:



Tujuan menerapkan Single Responsibility adalah jika ingin melakukan perubahan pada sebuah *class* yang memiliki tanggung jawab yang banyak, perubahan yang akan dilakukan berpotensi untuk mempengaruhi fungsionalitas dan tanggung jawab lain yang saling berkaitan di dalam *class* tersebut.

Pemisahan tanggung jawab yang baik hanya bisa dilakukan ketika sebuah gambaran lengkap aplikasi secara keseluruhan telah dipahami dengan baik. Hal ini berisi tentang bagaimana aplikasi tersebut dapat bekerja, sehingga kita dapat memisahnya dengan rinci.

### 2.1.3 Open/Close Principle (OCP)

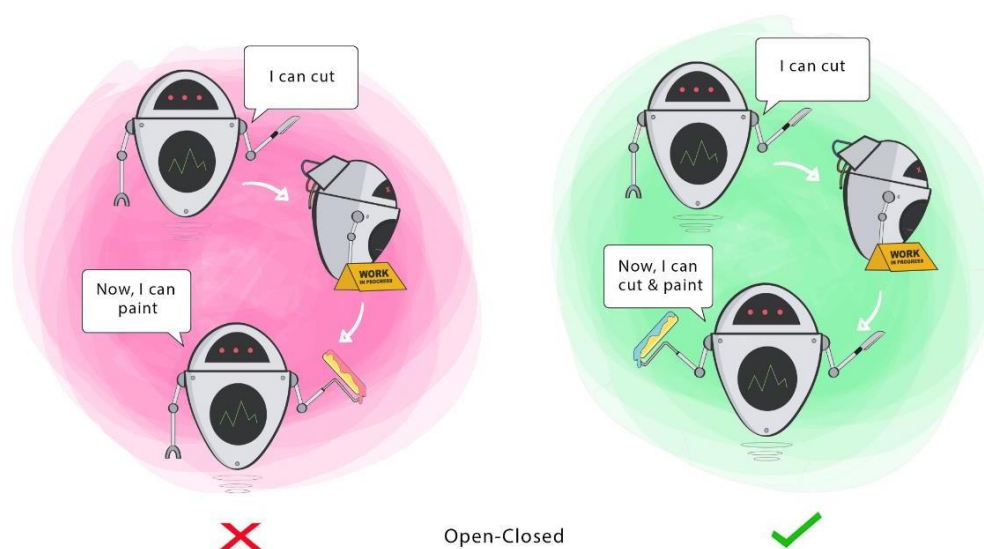
Open Principle adalah keadaan ketika sebuah sistem dapat ditambahkan dengan spesifikasi baru yang dibutuhkan. Sedangkan Close Principle adalah agar ketika ingin



menambahkan spesifikasi baru, kita tidak perlu mengubah atau memodifikasi sistem yang telah ada.

Aturan ini sekilas terlihat bertentangan satu sama lain. Namun ketika bisa mengatur dependensi sistem dengan baik dan benar, dengan mudahnya aturan tersebut dapat dicapai. Secara umum, penggunaan aturan open/close diterapkan dengan memanfaatkan *interface* dan abstraksi kelas daripada menggunakan sebuah kelas konkret. Penggunaan *interface* dan abstraksi kelas bertujuan agar dapat mudah diperbaiki setelah pengembangan tanpa harus mengganggu kelas yang mewarisi dan ketika ingin membuat fungsionalitas baru, cukup dengan membuat kelas baru dan mewarisi *interface* atau abstraksi tersebut.

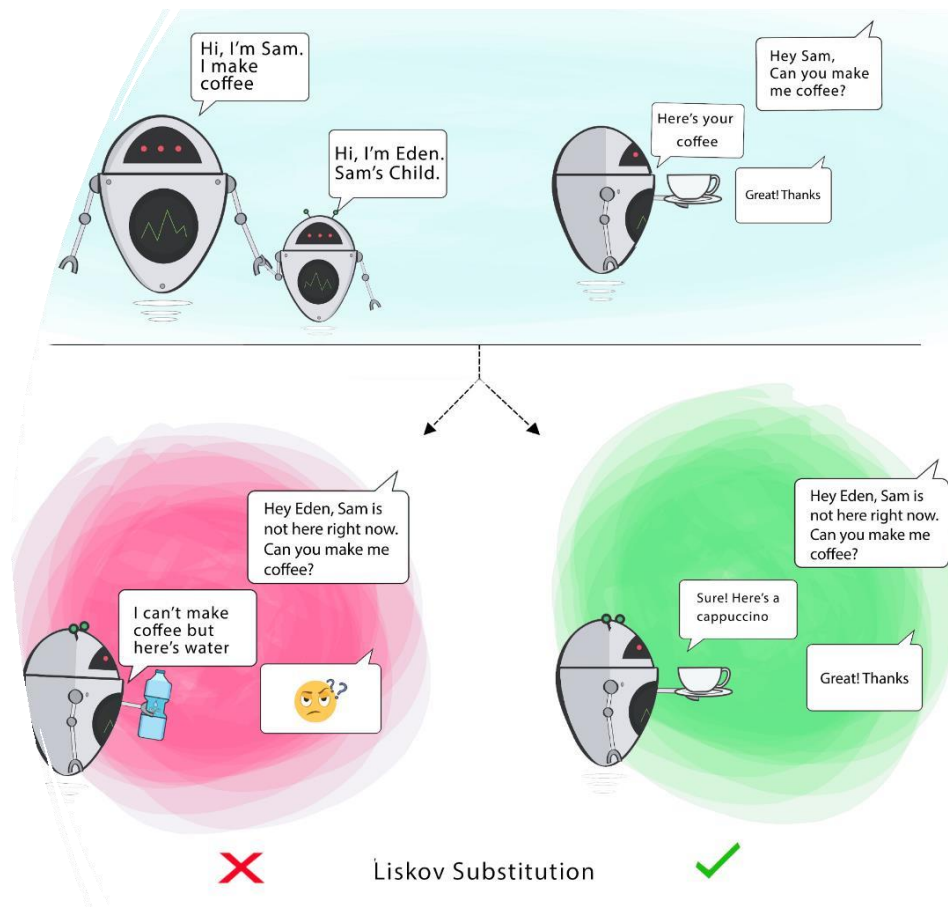
Berikut adalah gambaran ilustrasi dari OCP:



### 2.1.4 Liskov Substitution Principle (LSP)

Sederhananya, Liskov's substitution adalah aturan yang berlaku untuk hirarki pewarisan. Hal ini mengharuskan untuk mendesain kelas-kelas yang dimiliki sehingga ketergantungan antar klien dapat disubstitusikan tanpa klien mengetahui tentang perubahan yang ada. Oleh karena itu, seluruh SubClass setidaknya dapat berjalan dengan cara yang sama seperti SuperClass-nya.

Berikut adalah gambaran ilustrasi dari LSP:



Untuk menjadikan sebuah kelas benar-benar menjadi SubClass, kelas tersebut tidak hanya wajib untuk menerapkan fungsi dan properti dari SuperClass, melainkan juga harus memiliki perilaku yang sama dengan SuperClass-nya. Untuk mencapainya, terdapat beberapa aturan yang harus dipatuhi.

## 2.1.4.1 Contravariant dan Covariant

Aturan pertama, SubClass harus memiliki sifat contravariant dan covariant. **Contravariant** adalah kondisi di mana parameter dari sebuah fungsi yang berada pada SubClass harus memiliki tipe dan jumlah yang sama dengan fungsi yang berada pada SuperClass-nya. Sedangkan **Covariant** adalah kondisi pengembalian nilai dari fungsi yang berada pada SubClass dan SuperClass.

## 2.1.4.2 Preconditions dan Postconditions

Selanjutnya adalah aturan preconditions dan postconditions. Ini merupakan tindakan yang harus ada sebelum atau sesudah sebuah proses dijalankan. Contohnya, ketika kita ingin



memanggil sebuah fungsi yang digunakan untuk membaca data dari database, terlebih dahulu kita harus memastikan database tersebut dalam keadaan terbuka agar proses dapat dijalankan. Ini disebut sebagai **precondition**. Sedangkan **postcondition**, contohnya saat proses baca tulis di dalam database telah selesai, kita harus memastikan database tersebut sudah tertutup.

#### **2.1.4.3 Invariant**

Berikutnya adalah invariant. Dalam pembuatan sebuah SubClass, SubClass tersebut harus memiliki invariant yang sama dengan SuperClass-nya. **Invariant** sendiri adalah penjelasan dari kondisi suatu proses yang benar sebelum proses tersebut dimulai dan tetap benar setelahnya.

#### **2.1.4.4 Constraint**

Terakhir, aturan tentang constraint dari sebuah SubClass. Secara default, SubClass dapat memiliki fungsi dan properti dari SuperClass-nya. Selain itu juga dapat menambahkan member baru di dalamnya. **Constraint** di sini adalah pembatasan yang ditentukan oleh SuperClass terhadap perubahan keadaan sebuah obyek. Sebagai contoh misal SuperClass memiliki obyek yang memiliki nilai tetap, maka SubClass tidak diijinkan untuk mengubah keadaan dari nilai obyek tersebut.

### **2.2 Latihan**

Tulis ulang dan coba code-code dalam Latihan berikut. Amati dan lakukan analisis terhadap perubahan yang dilakukan.

#### **2.2.1 Latihan SRP**

Perhatikan studi kasus pada order (pesanan) sebuah mesin kasir atau POS (Point Of Sales) yang dimodelkan menjadi code class Order berikut:



## MODUL IMPLEMENTASI PERANGKAT LUNAK

OLEH : Tim Dosen

### Order

- + calculateTotalSum
- + getItems()
- + getItemCount()
- + addItem()
- + deleteItem()
- + printOrder()
- + showOrder()
- + getDailyHistory()
- + getMonthlyHistory()

```
class Order {
    void calculateTotalSum() {
        /*...*/
    }
    void getItems() {
        /*...*/
    }
    void getItemCount() {
        /*...*/
    }
    void addItem(Item item) {
        /*...*/
    }
    void deleteItem(Item item) {
        /*...*/
    }
    void printOrder() {
        /*...*/
    }
    void showOrder() {
        /*...*/
    }
    void getDailyHistory() {}
    void getMonthlyHistory() {}
}
```

Dengan menerapkan prinsip dari SRP, ubah code diatas menjadi code berikut:



## MODUL IMPLEMENTASI PERANGKAT LUNAK

OLEH : Tim Dosen

### Order

- + calculateTotalSum
- + getItems()
- + getItemCount()
- + addItem()
- + deleteItem()

### OrderViewer

- + printOrder(order: Order)
- + showOrder(order: Order)

### OrderHistory

- + getDailyHistory()
- + getMonthlyHistory()

### Class Order

```
class Order {  
    void calculateTotalSum() {  
    }  
  
    void getItems() {  
    }  
  
    void getItemCount() {  
    }  
  
    void addItem(Item item) {  
    }  
  
    void deleteItem(Item item) {  
    }  
}
```

### Class Order History

```
class OrderHistory {  
    void getDailyHistory() {}  
    void getMonthlyHistory() {}  
}
```

### Class OrderViewer

```
class OrderViewer {  
    void printOrder(Order order) {  
    }  
  
    void showOrder(Order order) {  
    }  
}
```

### Class Main

```
class Main {  
    public static void main(String[] args) {  
        Item item = new Item();  
    }  
}
```





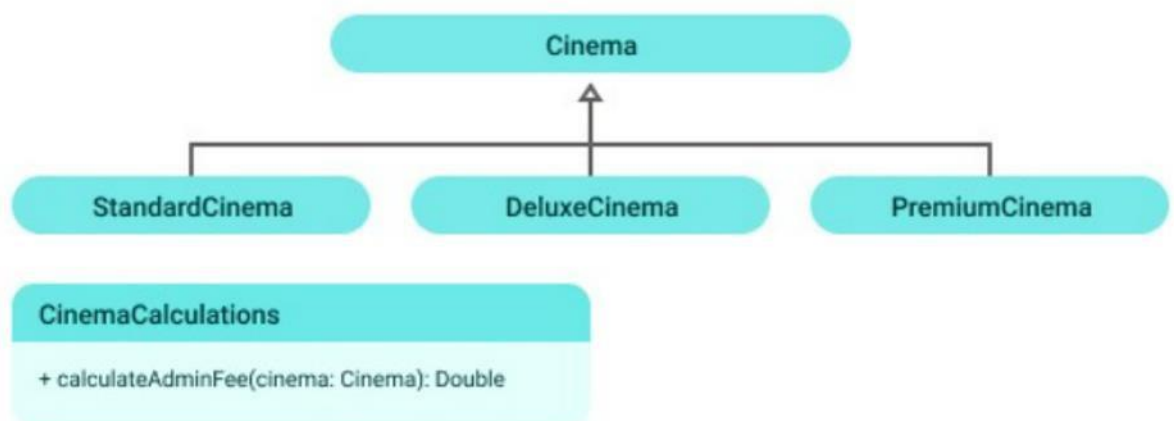
```

    Order order = new Order();
    order.addItem(item);
    OrderHistory history = new OrderHistory();
    history.getDailyHistory();
    OrderViewer viewer = new OrderViewer();
    viewer.printOrder(order);
}
}

```

### 2.2.2 Latihan Open/Close Principle

Perhatikan contoh kasus perhitungan biaya admin dari berbagai jenis bioskop berikut. Terdapat berbagai jenis bioskop dengan fasilitas dan biaya admin yang berbeda-beda. Bioskop dimodelkan menjadi Class Cinema, sedangkan perhitungan biaya admin dimodelkan menjadi Class CinemaCalculations.



Class Cinema

```

class Cinema {
    public Double price;
}

```

Class StandardCinema

```

class StandardCinema extends Cinema {
    public StandardCinema(double price) {
        this.price = price;
    }
}

```

Class DeluxeCinema

```

class DeluxeCinema extends Cinema {
    public DeluxeCinema(double price) {
        this.price = price;
    }
}

```





### Class PremiumCinema

```
class PremiumCinema extends Cinema {  
    public PremiumCinema(double price) {  
        this.price = price;  
    }  
}
```

### Class CinemaCalculations

```
class CinemaCalculations {  
    public Double calculateAdminFee(Cinema cinema) {  
        if (cinema instanceof StandardCinema) {  
            return ((StandardCinema) cinema).price * 10 / 100;  
        } else if (cinema instanceof DeluxeCinema) {  
            return ((DeluxeCinema) cinema).price * 12 / 100;  
        } else if (cinema instanceof PremiumCinema) {  
            return ((PremiumCinema) cinema).price * 20 / 100;  
        } else  
            return 0.0;  
    }  
}
```

Dengan penerapan OCP, code diatas diubah menjadi code berikut:

### Abstract Class Cinema

```
abstract class Cinema {  
    public Double price;  
    abstract Double calculateAdminFee();  
}
```

### Class StandardCinema

```
class StandardCinema extends Cinema {  
    public StandardCinema(Double price) {  
        this.price = price;  
    }  
  
    @Override  
    Double calculateAdminFee() {  
        return price * 10 / 100;  
    }  
}
```

### Class DeluxeCinema

```
class DeluxeCinema extends Cinema {  
    public DeluxeCinema(Double price) {  
        this.price = price;  
    }  
  
    @Override
```



## MODUL IMPLEMENTASI PERANGKAT LUNAK

OLEH : Tim Dosen

```
Double calculateAdminFee() {  
    return price * 12 / 100;  
}
```

### Class PremiumCinema

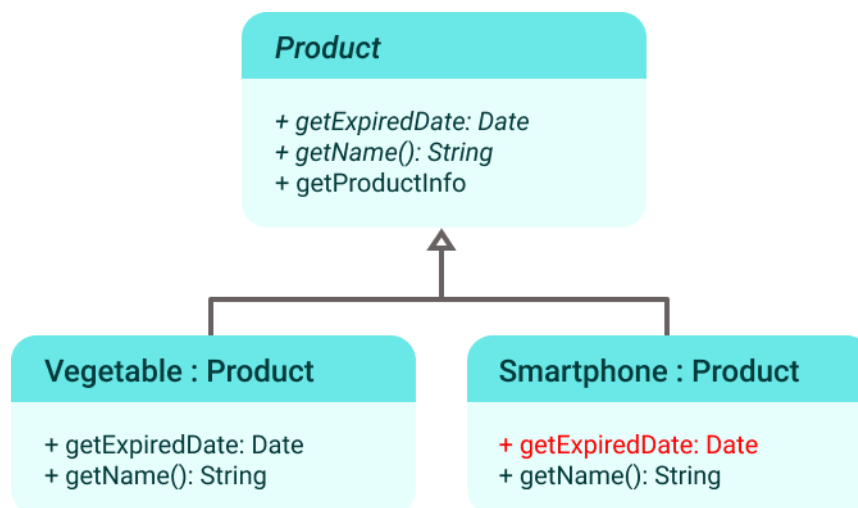
```
class PremiumCinema extends Cinema {  
    public PremiumCinema(Double price) {  
        this.price = price;  
    }  
  
    @Override  
    Double calculateAdminFee() {  
        return price * 20 / 100;  
    }  
}
```

### Class Main

```
class Main {  
    public static void main(String[] args) {  
        StandardCinema standardCinema = new StandardCinema(100.0);  
        Double adminFee = standardCinema.calculateAdminFee();  
        System.out.println(adminFee);  
    }  
}
```

### 2.2.3 Latihan LSP

Berikut adalah diagram dan contoh script code.





### Class Product

```
abstract class Product {
    abstract String getName();
    abstract Date getExpiredDate();

    /**
     * Function to get all of information about product
     */
    public void getProductInfo() {
        // some magic code
    }
}
```

### Class Vegetable

```
class Vegetable extends Product {

    @Override
    String getName() {
        return "Broccoli";
    }

    @Override
    Date getExpiredDate() {
        return new Date();
    }
}
```

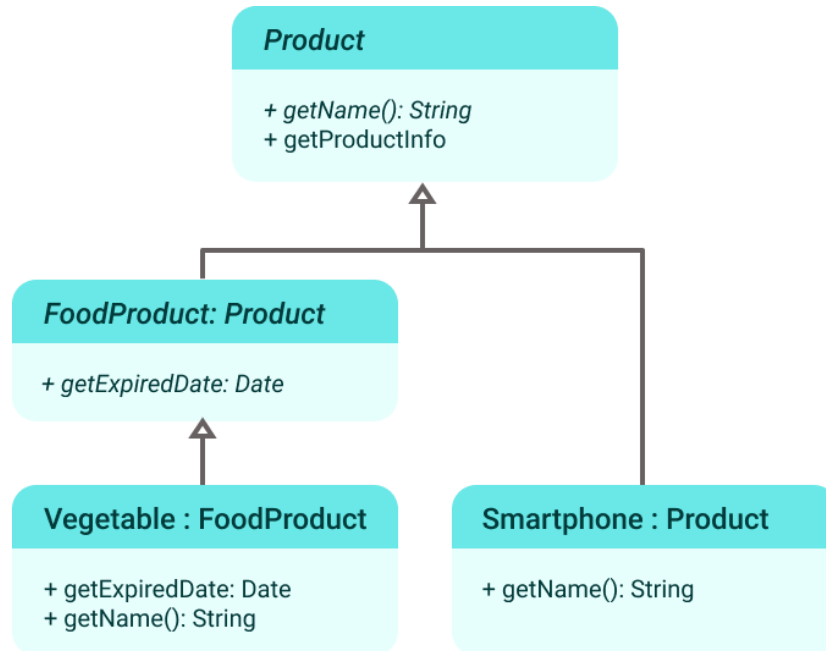
### Class Smartphone

```
class Smartphone extends Product {

    @Override
    String setName() {
        return "Samsung S10+ Limited Edition";
    }

    @Override
    Date setExpiredDate() {
        return new Date(); // ???????
    }
}
```

Untuk penerapan Principle Liskov Substitution, ubahlah code diatas menjadi code berikut:



### Class Product

```
abstract class Product {
    abstract String getName();

    /**
     * Function to get all of information about product
     */
    public void getProductInfo() {
        // some magic code
    }
}
```

### Class FoodProduct

```
abstract class FoodProduct extends Product {
    abstract Date getExpiredDate();
}
```

### Class Vegetable

```
class Vegetable extends FoodProduct {

    @Override
    String getName() {
        return "Broccoli";
    }

    @Override
    Date getExpiredDate() {
        return new Date();
    }
}
```



## MODUL IMPLEMENTASI PERANGKAT LUNAK

OLEH : Tim Dosen

```
}  
}
```

### Class Smartphone

```
class Smartphone extends Product {  
    @Override  
    String getName() {  
        return "Samsung S10+ Limited Edition";  
    }  
}
```

## 2.3 Tugas

Untuk memahami principle dari SOLID terutama bagian Single Responsibility, Open/Closed dan Liskov Substitution. Selesaikan perintah berikut.

1. Pada Latihan 3.2.1 Single Responsibility, Jelaskan alasan kenapa bentuk/design codenya harus diubah?
2. Pada Latihan 3.2.2 Open/Closed, Jelaskan alasan kenapa bentuk/design codenya harus diubah?
3. Buatlah Class CinemaMahasiswa yang biaya adminnya adalah 5% dari harga umum (5/100).
4. Pada Latihan 3.2.3 Liskov Substitution, Jelaskan alasan kenapa bentuk/design codenya harus diubah?